Semantic Validator Rules Pack v0.1

A polished reading edition prepared from the source Markdown, preserving the original technical content while improving hierarchy, print readability, and presentation.

AUTHOR

Kotov Ivan
Bruxelles, 2026

STATUS

Draft semantic validator rules pack

SCOPE

Semantic validation layer over `Schema Pack v0.1`

PURPOSE

- structurally valid objects,
- semantically coherent objects,
- forbidden state combinations,
- illegal cross-object relations,
- and transition attempts that must fail before execution or projection.

ExecutionNode · GlitchNode · ResearchNode · BackwardNode · EvidenceRecord · ChallengeRecord · ReviewRecord · TransitionGuard · ReopenabilityGate · GraphNodeView · GraphEdgeView · GraphSlice · StatusTuple

SEMANTIC RULES SPECIFICATION

Status: Draft semantic validator rules pack
Scope: Semantic validation layer over `Schema Pack v0.1`
Purpose: Semantic constraints that prevent structurally valid objects from passing as semantically legitimate system states.

---

1. Why this pack exists

A schema can prove that: - fields exist, - enums are correct, - nullability is respected, - references are shaped correctly.

That is necessary.

It is not enough.

A system can be perfectly schema-valid and still be semantically dishonest.

Examples: - a `ResearchNode` marked executable, - a `GlitchNode` without a real collision source, - a `GraphNodeView` treated as authoritative, - a signed evidence record treated as if signature implied legitimacy, - a settled challenge with no review outcome, - a reopened research branch with no new evidence.

This pack exists to prevent that.

It is the layer where the system says:

“Your object is well-formed, but it still does not make sense.”

---

2. Validation stack position

The validation stack should be understood as:

1. **Schema validation**
   - shape
   - required fields
   - basic type correctness
2. **Semantic validation**
   - internal logical coherence

- allowed combinations
- cross-object meaning

3. **Transition validation**
   - whether movement from one semantic state to another is legal
4. **Integrity validation**
   - hash, witness, signature, lineage correctness

This document defines level 2.

---

3. Validation result model

Future validators should return explicit, typed results.

Suggested result shape:

```
{
  "ok": false,
  "layer": "semantic",
  "rule_code": "SEM_RESEARCH_EXECUTABLE_FORBIDDEN",
  "severity": "error",
  "target_ref": {"node_id": "research_01H...", "node_kind": "ResearchNode"},
  "message": "ResearchNode cannot be executable.",
  "details": {
    "lane": "research",
    "executable": true
  }
}
```

Required result fields

- `ok`
- `layer`
- `rule_code`
- `severity`
- `message`

Optional fields

- `target_ref`
- `details`
- `hint`
- `related_refs`

---

4. Severity classes

```
error     -> object must be rejected
warning   -> object may be accepted for inspection but not for execution or stable
projection
notice    -> object is valid but degraded / caution-bearing
```

Rule

Semantic validator rules in this pack are primarily: - `error` - some `warning` - very few `notice`

Do not overuse notice. If a combination is actually dangerous, it should fail.

---

5. Foundational semantic rules for `StatusTuple`

5.1 Runtime lane executable consistency

Rule code

`SEM_RUNTIME_EXECUTABLE_MISMATCH`

Condition

If: - `lane == "runtime"` - `active == true`

then: - `evidence_state != "cinematic_only"`

Why

Runtime objects may be blocked, refused, or expired. They may not be reduced to cinematic-only representation.

5.2 Cinematic lane execution prohibition

Rule code

`SEM_CINEMATIC_EXECUTABLE_FORBIDDEN`

Condition

If: - `lane == "cinematic"`

then: - `executable == false`

Severity

`error`

5.3 Cinematic evidence-lane consistency

Rule code

`SEM_CINEMATIC_STATE_LANE_MISMATCH`

Condition

If: - `evidence_state == "cinematic_only"`

then: - `lane == "cinematic"`

Severity

`error`

5.4 Reopenability lane restriction

Rule code

`SEM_REOPENABILITY_LANE_FORBIDDEN`

Condition

If: - `reopenability != null`

then: - `lane in {"research", "historical"}`

Severity

`error`

5.5 Executable lane restriction

Rule code

`SEM_EXECUTABLE_LANE_FORBIDDEN`

Condition

If: - `executable == true`

then: - `lane == "runtime"`

Severity

`error`

5.6 Challenge/evidence consistency

Rule code

`SEM_CHALLENGE_EVIDENCE_MISMATCH`

Condition

If: - `challenge_status != null`

then: - `evidence_state in {"witnessed", "signed", "challenge_open", "settled", "expired"}`

Severity

`error`

Why

A challenge cannot meaningfully attach to a purely cinematic or weakly ungrounded object unless the system explicitly allows speculative discussion. For default governance semantics, challenge presupposes some evidentiary standing.

5.7 Expired executable contradiction

Rule code

`SEM_EXPIRED_EXECUTABLE_FORBIDDEN`

Condition

If: - `expired == true`

then: - `executable == false`

Severity

`error`

---

6. Semantic rules for `ExecutionNode`

6.1 Runtime lane requirement

Rule code

`SEM_EXECUTIONNODE_NONRUNTIME_LANE`

Condition

`ExecutionNode.status.lane != "runtime"`

Severity

`error`

6.2 Execution node cannot be cinematic-only

Rule code

`SEM_EXECUTIONNODE_CINEMATIC_ONLY_FORBIDDEN`

Condition

`ExecutionNode.status.evidence_state == "cinematic_only"`

Severity

`error`

6.3 End timestamp ordering

Rule code

`SEM_EXECUTIONNODE_TIME_ORDER_INVALID`

Condition

If `ended_ts != null` and `ended_ts < started_ts`

Severity

`error`

---

7. Semantic rules for `GlitchNode`

7.1 Source kind requirement

Rule code

`SEM_GLITCH_SOURCE_KIND_INVALID`

Condition

`source_execution_ref.node_kind != "ExecutionNode"`

Severity

`error`

7.2 Runtime lane requirement

Rule code

`SEM_GLITCH_NONRUNTIME_LANE`

Condition

`status.lane != "runtime"`

Severity

`error`

7.3 Glitch cannot be executable

Rule code

`SEM_GLITCH_EXECUTABLE_FORBIDDEN`

Condition

`status.executable == true`

Severity

`error`

7.4 Witness requirement for elevated evidence states

Rule code

`SEM_GLITCH_WITNESS_REQUIRED`

Condition

If `status.evidence_state in {"witnessed", "signed", "challenge_open", "settled"}` and `witness_ref == null`

Severity

`error`

7.5 Challenge window requirement

Rule code

`SEM_GLITCH_CHALLENGE_WINDOW_REQUIRED`

Condition

If `status.challenge_status != null` and `challenge_window == null`

Severity

`error`

7.6 Collision meaning requirement

Rule code

`SEM_GLITCH_REASON_CODE_REQUIRED`

Condition

`reason_code` empty or whitespace-only

Severity

`error`

Why

A `GlitchNode` without a reason code is just an atmospheric failure. That is not enough.

---

8. Semantic rules for `ResearchNode`

8.1 Source kind requirement

Rule code

`SEM_RESEARCH_SOURCE_KIND_INVALID`

Condition

`source_glitch_ref.node_kind != "GlitchNode"`

Severity

`error`

8.2 Research lane requirement

Rule code

`SEM_RESEARCH_NONRESEARCH_LANE`

Condition

`status.lane != "research"`

Severity

`error`

8.3 Executability prohibition

Rule code

`SEM_RESEARCH_EXECUTABLE_FORBIDDEN`

Condition

`status.executable == true`

Severity

`error`

8.4 Reopenability presence requirement

Rule code

`SEM_RESEARCH_REOPENABILITY_REQUIRED`

Condition

`status.reopenability == null`

Severity

`error`

8.5 Missing-evidence semantic minimum

Rule code

`SEM_RESEARCH_EMPTY_NEEDS_WARNING`

Condition

Both: - `missing_evidence` empty - `required_resources` empty

Severity

`warning`

Why

A research object with neither missing evidence nor required resources may still be valid, but it risks becoming vague speculation.

8.6 Witness downgrade caution

Rule code

`SEM_RESEARCH_WITNESS_OVERCLAIM`

Condition

If: - `witness_ref != null` - `status.evidence_state == "asserted"`

Severity

`warning`

Why

Possible, but suspicious. Usually witness presence should imply stronger evidentiary standing or clear explanation in metadata.

---

9. Semantic rules for `BackwardNode`

9.1 Source kind requirement

Rule code

`SEM_BACKWARD_SOURCE_KIND_INVALID`

Condition

`source_research_ref.node_kind != "ResearchNode"`

Severity

`error`

9.2 Research lane requirement

Rule code

`SEM_BACKWARD_NONRESEARCH_LANE`

Condition

`status.lane != "research"`

Severity

`error`

9.3 Executability prohibition

Rule code

`SEM_BACKWARD_EXECUTABLE_FORBIDDEN`

Condition

`status.executable == true`

Severity

`error`

9.4 Empty gap statement prohibition

Rule code

`SEM_BACKWARD_GAP_REQUIRED`

Condition

`gap_statement` empty or whitespace-only

Severity

`error`

---

10. Semantic rules for `EvidenceRecord`

10.1 Signed requires signer

Rule code

`SEM_EVIDENCE_SIGNER_REQUIRED`

Condition

If `evidence_state == "signed"` and `signer == null`

Severity

`error`

10.2 Witnessed-or-beyond requires witness ref

Rule code

`SEM_EVIDENCE_WITNESS_REF_REQUIRED`

Condition

If `evidence_state in {"witnessed", "signed", "challenge_open", "settled"}` and `witness_ref == null`

Severity

`error`

10.3 Cinematic-only evidence target restriction

Rule code

`SEM_EVIDENCE_CINEMATIC_TARGET_FORBIDDEN`

Condition

If `evidence_state == "cinematic_only"` and target refers to runtime or research object

Severity

`error`

Why

`cinematic_only` is a display-state, not a runtime/research truth-state.

10.4 Signature legitimacy shortcut prohibition

Rule code

`SEM_EVIDENCE_SIGNATURE_NOT_AUTHORITY`

Condition

If metadata or downstream flag claims: - `legitimate == true` only because `evidence_state == "signed"`

Severity

`error`

Why

Signature proves integrity of record, not legitimacy of action.

---

11. Semantic rules for `ChallengeRecord`

11.1 Deadline ordering

Rule code

`SEM_CHALLENGE_DEADLINE_INVALID`

Condition

`deadline_ts < opened_ts`

Severity

`error`

11.2 Empty reason prohibition

Rule code

`SEM_CHALLENGE_REASON_REQUIRED`

Condition

`reason` empty or whitespace-only

Severity

`error`

11.3 Archived-without-resolution warning

Rule code

`SEM_CHALLENGE_ARCHIVE_WITHOUT_LINEAGE`

Condition

If `status == "archived"` and no review / closure lineage exists

Severity

`warning`

11.4 Target admissibility requirement

Rule code

`SEM_CHALLENGE_TARGET_NOT_CHALLENGEABLE`

Condition

Target object does not meet challengeability rules defined elsewhere

Severity

`error`

---

12. Semantic rules for `ReviewRecord`

12.1 Reclassify class fields requirement

Rule code

`SEM_REVIEW_RECLASS_FIELDS_REQUIRED`

Condition

If `outcome == "reclassify"` and either class field is null

Severity

`error`

12.2 Signed review requires witness ref

Rule code

`SEM_REVIEW_SIGNED_WITNESS_REQUIRED`

Condition

If `signed == true` and `witness_ref == null`

Severity

`error`

12.3 Empty notes warning

Rule code

`SEM_REVIEW_EMPTY_NOTES_WARNING`

Condition

`notes` empty or whitespace-only

Severity

`warning`

Why

Possible, but poor review hygiene.

12.4 Branch split lineage requirement

Rule code

`SEM_REVIEW_SPLIT_LINEAGE_REQUIRED`

Condition

If `outcome == "branch_split"` and no split lineage metadata exists

Severity

`error`

---

13. Semantic rules for `TransitionGuard`

13.1 Allowed false with empty reason prohibition

Rule code

`SEM_GUARD_REASON_REQUIRED`

Condition

If `allowed == false` and `reason` empty

Severity

`error`

13.2 Legal shortcut prohibition

Rule code

`SEM_GUARD_SHORTCUT_INVALID`

Condition

If `allowed == true` but transition violates known forbidden combinations

Severity

`error`

Why

A `TransitionGuard` is not allowed to “bless” forbidden transitions.

---

14. Semantic rules for `ReopenabilityGate`

14.1 Source kind requirement

Rule code

`SEM_GATE_SOURCE_KIND_INVALID`

Condition

`research_ref.node_kind != "ResearchNode"`

Severity

`error`

14.2 Allowed-without-basis warning

Rule code

`SEM_GATE_ALLOWED_WITHOUT_BASIS`

Condition

If: - `allowed == true` - `required_evidence` empty - `required_review == false` - `current_state != "reopenable"`

Severity

`warning`

Why

Possible in some edge cases, but likely a policy leak.

---

15. Semantic rules for graph/read objects

15.1 Graph node executable field prohibition

Rule code

`SEM_GRAPH_EXECUTABLE_FIELD_FORBIDDEN`

Condition

A `GraphNodeView` contains runtime-authority field such as `executable`

Severity

`error`

Why

Graph view is projection, not authority carrier.

15.2 Graph/runtime lane mismatch warning

Rule code

`SEM_GRAPH_LANE_BADGE_MISMATCH`

Condition

`GraphNodeView.lane` and `badge.evidence_state` imply contradictory semantics

Examples: - lane = `runtime` - badge = `cinematic_only`

Severity

`error`

15.3 Graph edge cinematic laundering prohibition

Rule code

`SEM_GRAPH_EDGE_LAUNDERING_FORBIDDEN`

Condition

An edge of kind `cinematic_projects` is rendered or exported as if it were: - `execution_flow` - `review_resolves` - `witness_binds`

Severity

`error`

15.4 Graph slice integrity warning

Rule code

`SEM_GRAPH_SLICE_MISSING_INTEGRITY_NOTICE`

Condition

Audit-mode graph slice has no `integrity_root`

Severity

`notice`

Why

Not always fatal, but worth flagging in serious audit contexts.

---

16. Cross-object semantic rules

16.1 Runtime -> Research direct collapse prohibition

Rule code

`SEM_RUNTIME_TO_RESEARCH_BYPASS_GLITCH`

Condition

A `ResearchNode` is derived directly from `ExecutionNode` without a `GlitchNode` intermediary

Severity

`error`

Why

This would erase the collision point and create hallucinated future continuity.

16.2 Research -> Runtime direct collapse prohibition

Rule code

`SEM_RESEARCH_TO_RUNTIME_BYPASS_REVIEW`

Condition

A `ResearchNode` or `BackwardNode` is used as executable runtime object without: - explicit reopenability approval, - evidence basis, - and legal transition path.

Severity

`error`

16.3 Challenge settlement without review prohibition

Rule code

`SEM_CHALLENGE_SETTLED_WITHOUT_REVIEW`

Condition

Challenge status resolved/settled but no linked `ReviewRecord`

Severity

`error`

16.4 Review rewrite without preserved lineage

Rule code

`SEM_REVIEW_LINEAGE_ERASURE_FORBIDDEN`

Condition

Review modifies or reclassifies target with no preserved prior lineage reference

Severity

`error`

16.5 Cinematic projection into runtime truth prohibition

Rule code

`SEM_CINEMATIC_TO_RUNTIME_FORBIDDEN`

Condition

Graph/cinematic object is fed back into runtime truth layer as if it were: - execution fact - witness fact - or legitimacy carrier

Severity

`error`

---

17. Suggested validator API shape

Future semantic validator API could look like:

```
def validate_semantics(obj: dict, kind: str, context: dict | None = None) ->
list[SemanticValidationResult]:
    ...
```

Where: - `kind` = schema/object kind - `context` may include: - related refs - graph lineage - evidence lookup - current lane policy - review lineage

Output

A list of: - zero or more errors - zero or more warnings - zero or more notices

Acceptance policy: - any `error` → reject - warnings may be accepted only for read-side or quarantine-side objects - notices do not block by default

---

18. Suggested rule-group organization

Group A — status coherence

- lane / executable / challenge / evidence consistency

Group B — object identity coherence

- source kinds
- target kinds
- required relation anchors

Group C — evidence coherence

- witness/signature/evidence-state logic

Group D — review coherence

- challenge deadlines
- review outcomes
- split lineage
- settlement logic

Group E — projection safety

- graph node / edge / slice cannot impersonate runtime truth

Group F — anti-collapse rules

- research cannot silently become runtime
- cinematic cannot silently become evidence
- signature cannot silently become legitimacy

---

19. Sample semantic failure set

Example

A `ResearchNode` is stored with: - `lane = "research"` - `executable = true` - `reopenability = null`

Expected semantic failures:

```
[
  {
    "ok": false,
    "layer": "semantic",
    "rule_code": "SEM_RESEARCH_EXECUTABLE_FORBIDDEN",
    "severity": "error",
    "message": "ResearchNode cannot be executable."
  },
  {
    "ok": false,
    "layer": "semantic",
    "rule_code": "SEM_RESEARCH_REOPENABILITY_REQUIRED",
    "severity": "error",
    "message": "ResearchNode must declare reopenability state."
  }
]
```

---

20. Explicit bridge

This pack adds the missing middle layer between: - schemas, - and transition guards.

It defines the meaning constraints that stop structurally valid objects from cheating.

That is the explicit bridge between: - object structure, - and trustworthy system behavior.

---

21. Hidden bridges

Hidden Bridge 1 — Cybernetics

Semantic rules preserve functional differentiation between regulator organs: collision, evidence, review, research, display.

Hidden Bridge 2 — Information Theory

Semantic rules prevent category collapse, which is one of the cheapest ways to destroy signal lineage while keeping syntax intact.

---

22. Earth paragraph

A real maintenance system may let you enter a signed inspection, a fault ticket, a provisional override, and a dashboard marker using the same keyboard. But that does not mean those entries are allowed to mean the same thing. If the system lets a signed note masquerade as a repaired machine, or a dashboard icon masquerade as a passed inspection, the paperwork will look immaculate right up until something catches fire. Semantic validation is the layer that says: “No. These are not the same event.”

23. Final position

`Semantic Validator Rules Pack v0.1` is where the stack stops being merely typed and starts becoming intellectually honest.

After this point, future implementation should not only ask: - “Is the object well-formed?”

It must also ask: - “Does this object have the right to mean what it claims to mean?”